

MERN STACK · INTERVIEW PREPARATION · 2026 EDITION

HTML

Basics to Advanced

Complete, in-depth notes — structure, semantics, forms, accessibility, HTML5 APIs, and React/JSX interoperability — built for real interviews and real development.

Prepared for Deepu Pandey | B.Tech CSE

Table of Contents

1. Introduction & Evolution of HTML — what HTML is, versions, how browsers use it

2. Document Structure & Anatomy — DOCTYPE, boilerplate, DOM tree

3. Elements, Tags & Attributes — block vs inline, void elements, global attributes

4. Semantic HTML5 — layout tags, why semantics matter

5. Text, Lists & Links

6. Images & Media — img, picture, srcset, audio, video

7. Tables

8. Forms & Validation — every input type, attributes, accessibility

9. The <head> — Meta Tags, SEO & Viewport

10. iframe, embed & object

11. HTML5 APIs for MERN developers — storage, canvas, geolocation, drag-drop

12. Accessibility (a11y) & ARIA

13. HTML Entities, Comments & Special Characters

14. HTML in React (JSX) — Key Differences

15. Performance & Best Practices (2026)

16. Master Differentiation Tables

17. Interview Question Bank (35+ Q&A)

18. Quick Revision Cheat-Sheet

01 Introduction & Evolution of HTML

1.1 What is HTML?

HTML (HyperText Markup Language) is the standard **markup language** used to create the structure and content of web pages. It is **not a programming language** — it has no logic, loops, or conditions. It only *describes* content using elements (tags), which the browser parses and renders visually.

- **HyperText** — text containing links to other text/documents (the "web" in World Wide Web).
- **Markup** — annotating content with tags that describe its meaning/structure, not how it looks (styling is CSS's job).

Interview one-liner: HTML is a markup language, not a programming language, because it lacks variables, loops, and conditional logic — it only structures content.

1.2 The Three Pillars of the Web

Layer	Purpose	Analogy
HTML	Structure & content	Skeleton of a body
CSS	Presentation & styling	Skin, clothes, appearance
JavaScript	Behavior & interactivity	Muscles/brain — makes it move and think

1.3 Brief History & Versions

Version	Year	Key Notes
HTML 1.0	1993	Very basic, no tables/forms
HTML 2.0	1995	Standardized forms
HTML 3.2	1997	Tables, applets
HTML 4.01	1999	CSS support, scripting, frames
XHTML	2000	HTML rewritten as strict XML (case-sensitive, self-closing tags)
HTML5	2014 (W3C), evolving as a "Living Standard" (WHATWG)	Semantic tags, audio/video, canvas, storage APIs, no plugins needed — current standard used in 2026

Interview: HTML5 is now maintained as a "**Living Standard**" by WHATWG (Web Hypertext Application Technology Working Group) — it has no fixed final version; it evolves continuously.

1.4 How a Browser Renders HTML (Critical Rendering Path)

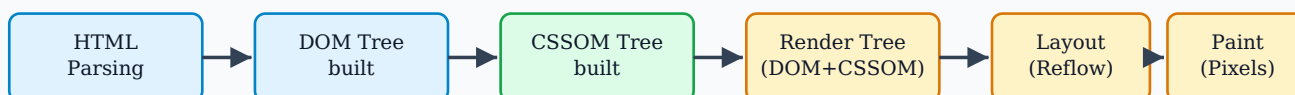


Fig 1.1 — Critical Rendering Path: HTML → DOM, CSS → CSSOM, combined → Render Tree → Layout → Paint on screen.

Interview Q: "Why is CSS placed in <head> and JS before </body>?"

A: CSSOM must be built before paint, so CSS is loaded early (in head) to avoid a flash of unstyled content (FOUC). JS can block HTML parsing (it's parser-blocking by default), so scripts are placed at the end of body (or loaded with `defer` / `async`) so the DOM builds first.

02 Document Structure & Anatomy

2.1 The Standard Boilerplate

```
<!-- Every HTML5 document starts like this -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Page Title</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello World</h1>
  <script src="app.js" defer</script>
</body>
</html>
```

2.2 Line-by-Line Meaning

Line	Meaning
<code><!DOCTYPE html></code>	Tells the browser to render in Standards Mode (HTML5). Without it, browsers use "Quirks Mode" (legacy, inconsistent rendering). It is NOT a tag — it's a directive, case-insensitive, no closing tag.
<code><html lang="en"></code>	Root element. <code>lang</code> helps screen readers pronounce content correctly and helps SEO/translation tools.
<code><head></code>	Metadata container — NOT rendered on page (title, meta, link, style, script-in-head go here).
<code><meta charset="UTF-8"></code>	Character encoding. UTF-8 supports virtually all world languages/emojis. Must be within first 1024 bytes.
<code><meta viewport...></code>	Controls how the page scales on mobile devices — essential for responsive design.
<code><body></code>	All visible content of the page lives here.

2.3 DOM (Document Object Model) Tree

The browser parses HTML into a tree of **nodes** called the DOM. JavaScript (and React under the hood) manipulates this tree to update the page.

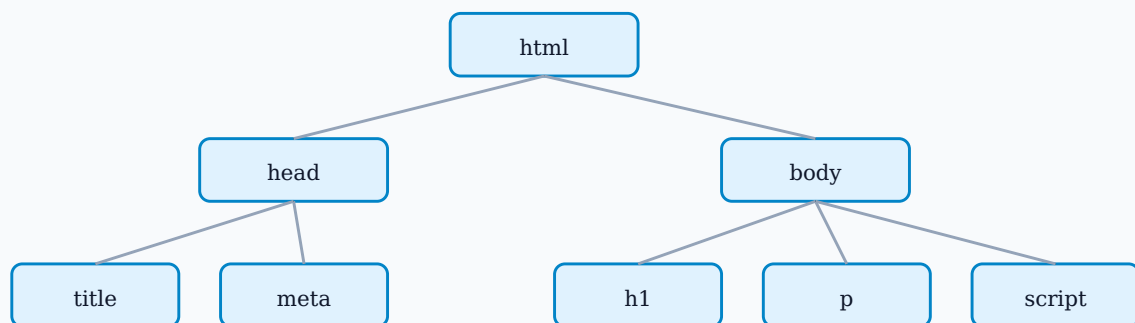


Fig 2.1 — Every tag becomes a node; nesting becomes parent-child relationships. This tree is what `document.querySelector` and React's Virtual DOM operate on.

2.4 HTML Comments

```
<!-- This is a comment, not rendered on the page -->
```

03 Elements, Tags & Attributes

3.1 Element vs Tag vs Attribute

Term	Meaning	Example
Tag	The markup keyword in angle brackets	<code><p></code> , <code></p></code>
Element	Opening tag + content + closing tag (complete unit)	<code><p>Hello</p></code>
Attribute	Extra info inside the opening tag, as <code>name="value"</code> pairs	<code></code>

3.2 Void (Self-Closing) Elements

Elements that **cannot have children/content** and need no closing tag:

`<img>` `<br>` `<hr>` `<input>` `<meta>` `<link>` `<source>` `<area>` `<col>` `<embed>` `<track>` `<wbr>`

Note: In plain HTML5 you don't need the trailing slash (`<br>` is valid), but in **JSX (React)** every void element MUST self-close: `<br />` , `<input />` — this is a common interview trap.

3.3 Block vs Inline vs Inline-Block Elements

Type	Behaviour	Examples
Block	Starts on a new line, takes full available width, accepts width/height/margin on all sides	<code>div</code> , <code>p</code> , <code>h1-h6</code> , <code>ul</code> , <code>li</code> , <code>form</code> , <code>section</code> , <code>header</code>
Inline	Flows within a line, only as wide as content, top/bottom margin/padding don't push other elements	<code>span</code> , <code>a</code> , <code>strong</code> , <code>em</code> , <code>img</code> , <code>label</code> , <code>input</code>
Inline-Block	Flows inline BUT respects width/height/margin (CSS-driven, e.g. <code>display:inline-block</code>)	Buttons styled via CSS

3.4 Global Attributes (work on almost every element)

Attribute	Purpose
<code>id</code>	Unique identifier — one per page, used by CSS/JS/anchors
<code>class</code>	Reusable identifier for CSS/JS — can repeat & multiple per element
<code>style</code>	Inline CSS (avoid in production — hard to maintain)
<code>title</code>	Tooltip text shown on hover
<code>data-*</code>	Custom data attributes for JS use, e.g. <code>data-user-id="42"</code>
<code>hidden</code>	Hides element (like <code>display:none</code>)
<code>contenteditable</code>	Makes element's text editable by the user
<code>draggable</code>	Enables HTML5 drag-and-drop
<code>tabindex</code>	Controls keyboard-tab focus order
<code>spellcheck</code>	Browser spell-checks the content

3.5 id vs class — Interview Favorite

Aspect	<code>id</code>	<code>class</code>
Uniqueness	Must be unique per page	Reusable across many elements
CSS Selector	<code>#idName</code>	<code>.className</code>
Specificity	Higher (100)	Lower (10)
JS Access	<code>getElementById()</code>	<code>getElementsByClassName()</code>
Typical Use	Unique hook, anchor links, one-off styling	Shared styling/components

04 Semantic HTML5

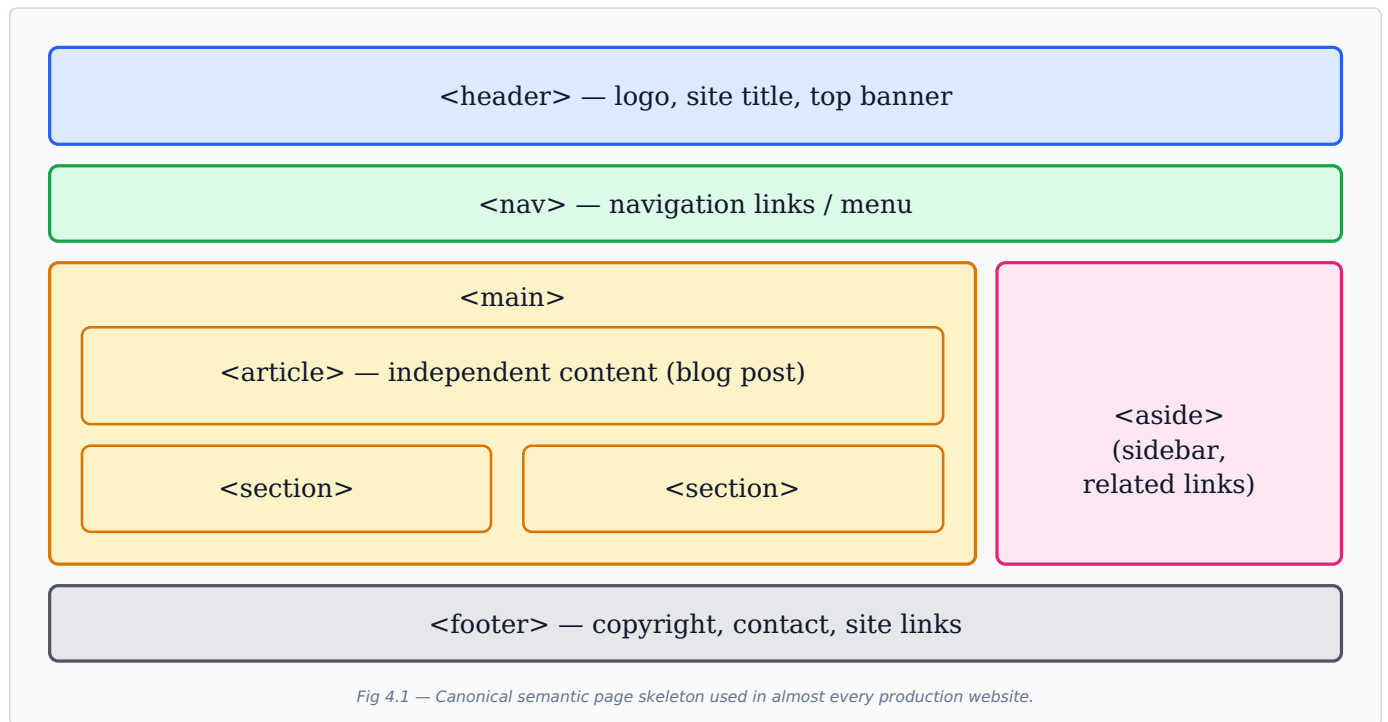
4.1 What Does "Semantic" Mean?

A semantic element clearly describes its **meaning/role** to both the browser and the developer — e.g. `<nav>` means navigation, unlike a generic `<div>` which means nothing.

4.2 Why Semantics Matter (Interview Gold)

- **Accessibility** — screen readers announce "navigation", "main content", "footer" so visually impaired users can jump between sections.
- **SEO** — search engine crawlers weigh content inside `<article>` / `<h1>` more meaningfully than inside generic `<div>` s.
- **Maintainability** — code is self-documenting; new developers understand layout instantly.
- **Consistent default styling/behaviour** hooks for browsers and tools.

4.3 Standard Semantic Layout



4.4 Key Semantic Tags Reference

Tag	Purpose
<code><header></code>	Introductory content / navigation for its nearest section or page
<code><nav></code>	Group of navigation links
<code><main></code>	The dominant, unique content of the document (only ONE per page)
<code><article></code>	Self-contained, independently distributable content (blog post, news story, forum post)
<code><section></code>	Thematic grouping of content, usually with a heading
<code><aside></code>	Content tangentially related to main content (sidebar, ads, pull-quotes)
<code><footer></code>	Footer for its nearest section or the whole page
<code><figure>/<figcaption></code>	Self-contained media (image/diagram) with a caption
<code><time></code>	Machine-readable date/time, e.g. <code><time datetime="2026-09-22">Sep 22</time></code>
<code><mark></code>	Highlighted/marked text
<code><details>/<summary></code>	Native collapsible/expandable widget — no JS needed
<code><dialog></code>	Native modal/popup dialog box

Interview Q: "Difference between `<section>` and `<div>` ?"

A: `<section>` carries semantic meaning (a thematic group, usually with its own heading) and is read by screen readers/SEO crawlers as a meaningful region. `<div>` is a generic, meaningless container used purely for styling/layout hooks.

Interview Q: "Can you have more than one `<main>`?" **A:** No — only one visible `<main>` per page.

05 Text, Lists & Links

5.1 Headings & Text Formatting

Tag	Use
<code><h1> ... <h6></code>	Headings, h1 = most important. Use only ONE h1 per page (SEO best practice); never skip levels.
<code><p></code>	Paragraph
<code></code>	Important text — bold AND semantic (screen readers emphasize)
<code></code>	Bold text — visual only, no semantic weight
<code></code>	Emphasized text — italic AND semantic (stressed tone)
<code><i></code>	Italic — visual only (e.g. foreign words, technical terms)
<code><small></code>	Fine print / side comments
<code></code> / <code><ins></code>	Deleted (strikethrough) / inserted (underline) text — used to show edits
<code><sub></code> / <code><sup></code>	Subscript (H ₂ O) / Superscript (x ²)
<code><blockquote></code>	Long quotation, block-level, often indented
<code><q></code>	Short inline quotation, browser adds quote marks automatically
<code><code></code> , <code><pre></code>	Inline code snippet; <code><pre></code> preserves whitespace/line breaks (used for code blocks)
<code><abbr title="..."></code>	Abbreviation with tooltip expansion
<code>
</code>	Line break (use sparingly — not for spacing, use CSS margin instead)
<code><hr></code>	Thematic break / horizontal rule

Interview Q: " vs , vs <i>?" **A:** The first pair (strong/em) carries semantic meaning for assistive tech and SEO; the second pair (b/i) is purely visual/presentational with no added meaning.

5.2 Lists

```
<ul>
  <li>Tea</li>
  <li>Coffee</li>
</ul>
```

```
<ol>
  <li>Wake up</li>
  <li>Code</li>
</ol>
```

`<ul>` = unordered (bullets), `<ol>` = ordered (numbers), `<li>` = list item (works inside both). `<dl>/<dt>/<dd>` = description list (term + definition pairs) — often overlooked but asked in interviews.

5.3 Links / Anchors

```
<a href="https://example.com" target="_blank" rel="noopener noreferrer">Visit</a>
<a href="#section2">Jump to Section 2</a>  <!-- anchor link -->
<a href="mailto:me@x.com">Email</a>
<a href="tel:+911234567890">Call</a>
```

Attribute	Purpose
<code>href</code>	Destination URL (absolute, relative, #id, mailto:, tel:)
<code>target="_blank"</code>	Opens link in a new tab
<code>rel="noopener noreferrer"</code>	Security must-know: prevents the new tab (opened via target=_blank) from accessing <code>window.opener</code> and hijacking the original page (reverse tabnabbing); noreferrer also hides referrer info.
<code>download</code>	Forces file download instead of navigation

06 Images & Media

6.1 The Tag

```

```

Attribute	Why it matters
<code>alt</code>	Mandatory for accessibility — read by screen readers; shown if image fails to load; heavily weighted by SEO/image search.
<code>width</code> / <code>height</code>	Reserves space before image loads → prevents Cumulative Layout Shift (CLS) , a Core Web Vital.
<code>loading="lazy"</code>	Native lazy loading — image loads only when near viewport, improves initial page performance.
<code>srcset</code> + <code>sizes</code>	Serve different image resolutions for different screen sizes (responsive images).

6.2 Responsive Images: <picture>

```
<picture>
  <source media="(max-width: 600px)" srcset="small.jpg">
  <source media="(min-width: 601px)" srcset="large.jpg">
   <!-- fallback -->
</picture>
```

The browser picks the best matching `<source>`; `<img>` inside is the required fallback. Used for art direction (different crops) or serving modern formats (WebP/AVIF) with fallback to JPEG.

6.3 Audio & Video

```
<video controls width="480" poster="thumb.jpg">
  <source src="movie.mp4" type="video/mp4">
  Your browser doesn't support video.
</video>

<audio controls autoplay loop muted>
  <source src="song.mp3" type="audio/mpeg">
</audio>
```

Key attributes: `controls` (show play/pause UI), `autoplay` (browsers require `muted` for autoplay to work), `loop`, `muted`, `preload` (none/metadata/auto).

HTML5's native `<audio>` / `<video>` removed the need for Flash plugins — a major reason HTML5 replaced HTML4.

07 Tables

7.1 Full Table Structure

```
<table>
  <caption>Student Marks</caption>
  <thead>
    <tr><th>Name</th><th>Marks</th></tr>
  </thead>
  <tbody>
    <tr><td>Deepu</td><td>95</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total</td><td>95</td></tr>
  </tfoot>
</table>
```

Tag	Role
<table>	Wrapper
<caption>	Table title, announced by screen readers
<thead>/<tbody>/<tfoot>	Semantic grouping of header/body/footer rows
<tr>	Table row
<th>	Header cell (bold, centered by default, <code>scope="col"/"/row"</code> for accessibility)
<td>	Data cell
colspan / rowspan	Merge cells across columns/rows

Interview trap: Never use `<table>` for page layout — it's semantically wrong and inaccessible. Use CSS Grid/Flexbox instead. Tables are ONLY for tabular data.

08 Forms & Validation

8.1 Basic Form Skeleton

```
<form action="/submit" method="POST" novalidate>
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required placeholder="you@mail.com">
  <button type="submit">Send</button>
</form>
```

Accessibility rule: Always pair `<label for="id">` with the matching `<input id="id">` — clicking the label focuses the input, and screen readers announce the label. This is one of the most commonly checked things in frontend interviews.

8.2 GET vs POST (very frequently asked)

Aspect	GET	POST
Data location	Appended to URL as query string	Sent in request body
Visibility	Visible in URL, browser history	Hidden from URL
Data limit	Limited (~2048 chars, URL length limit)	Large data / files allowed
Caching	Can be cached/bookmarked	Not cached by default
Idempotent?	Yes — safe to repeat	No — may create duplicate records
Typical use	Search, filters, fetching data	Login, file upload, creating/updating data

8.3 All <input> Types (HTML5)

Type	Purpose
<code>text</code>	Single-line free text
<code>password</code>	Masked input
<code>email</code>	Validates email format automatically; mobile keyboards show @ key
<code>number</code>	Numeric input with spinners; supports <code>min/max/step</code>
<code>tel</code>	Phone number (no built-in format validation; use <code>pattern</code>)
<code>url</code>	Validates URL format
<code>search</code>	Search box styling (adds a clear "x" in some browsers)
<code>date</code> , <code>time</code> , <code>datetime-local</code> , <code>month</code> , <code>week</code>	Native date/time pickers
<code>checkbox</code>	Multiple selection, boolean toggle
<code>radio</code>	Single choice among a group sharing the same <code>name</code>
<code>range</code>	Slider control (<code>min/max/step</code>)
<code>color</code>	Native color picker
<code>file</code>	File upload (<code>accept</code> , <code>multiple</code> attributes)
<code>hidden</code>	Invisible field sent with form (e.g. CSRF token)
<code>submit</code> / <code>reset</code> / <code>button</code>	Form action buttons

8.4 Other Form Elements

Tag	Purpose
<code><textarea></code>	Multi-line text input
<code><select>/<option></code>	Dropdown; <code><optgroup></code> groups options; <code>multiple</code> allows multi-select
<code><fieldset>/<legend></code>	Groups related fields with a visible caption — good for accessibility & radio groups
<code><datalist></code>	Autocomplete suggestions attached to an input via <code>list</code> attribute
<code><output></code>	Displays the result of a calculation/user action
<code><progress></code>	Task completion bar
<code><meter></code>	Scalar value within a known range (e.g. disk usage)

8.5 Built-in Validation Attributes

Attribute	Effect
<code>required</code>	Field must be filled before submit
<code>minlength</code> / <code>maxlength</code>	Character length limits
<code>min</code> / <code>max</code>	Numeric/date range limits
<code>pattern="regex"</code>	Custom regex validation, e.g. <code>pattern="[0-9]{10}"</code> for a 10-digit phone number
<code>step</code>	Increment interval for number/range/date
<code>novalidate</code>	(on <code><form></code>) disables native browser validation, e.g. when doing custom JS validation

Interview Q: "How does client-side validation differ from server-side?" **A:** Client-side (HTML5 attributes/JS) gives instant UX feedback but can be bypassed (disabled JS, direct API calls via Postman) — so **server-side validation is mandatory for security**; client-side is only for user experience.

8.6 multipart/form-data — MERN file uploads

```
<form action="/upload" method="POST" enctype="multipart/form-data">
  <input type="file" name="resume" accept=".pdf">
</form>
```

`enctype="multipart/form-data"` is mandatory whenever a form uploads files — plain URL-encoding can't carry binary data. On the Node/Express backend this is commonly parsed with **multer**.

09 The <head> — Meta Tags, SEO & Viewport

9.1 Essential Meta Tags

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="Short summary shown in Google search results">
<meta name="keywords" content="html, css, js"> <!-- largely ignored by Google now -->
<meta name="robots" content="index, follow">
<link rel="icon" href="favicon.ico">
<link rel="canonical" href="https://site.com/page">
```

9.2 Open Graph (Social Media Previews)

```
<meta property="og:title" content="My Page">
<meta property="og:description" content="Description for link previews">
<meta property="og:image" content="preview.jpg">
```

These tags control how a link looks when shared on WhatsApp, LinkedIn, Twitter/X, etc. — commonly asked in full-stack interviews about "how did you handle SEO/social sharing?"

9.3 script: defer vs async (High-frequency interview topic)

Normal <script>

HTML parsing

Fetch+Run JS (blocks)

HTML parsing resumes

<script async>

HTML parsing (continues) fetch JS (parallel)

run JS (parsing pauses briefly, runs ASAP, order NOT guaranteed)

<script defer>

HTML parsing completes fully (JS fetched in background, in order)

run JS

Fig 9.1 — Normal scripts block parsing; async runs as soon as downloaded (order not guaranteed); defer runs after parsing, in document order (best for most app scripts).

Attribute	Download	Execution	Order preserved?	Best for
(none)	Blocks HTML parsing	Immediately	Yes	Rarely — avoid in head
async	Parallel, non-blocking	As soon as ready (interrupts parsing)	No	Independent scripts (ads, analytics)
defer	Parallel, non-blocking	After DOM fully parsed	Yes	App scripts needing full DOM (most common choice)

10 iframe, embed & object

```
<iframe src="https://maps.google.com/..." width="600" height="400"
  loading="lazy" sandbox="allow-scripts" title="Google Map"></iframe>
```

Tag	Purpose
<iframe>	Embeds another full HTML document inside the current page (maps, YouTube embeds, payment widgets)
<embed>	Embeds external content/plugin resources (PDFs, media)
<object>	Similar to embed, more generic, supports fallback content between tags

Security note: `sandbox` attribute restricts what an iframe can do (block scripts, forms, popups) — crucial for embedding untrusted third-party content safely. This is a common security interview question in full-stack roles.

11 HTML5 APIs for MERN Developers

11.1 Web Storage: localStorage vs sessionStorage vs cookies

Feature	localStorage	sessionStorage	Cookies
Capacity	~5-10 MB	~5-10 MB	~4 KB
Expiry	Never (until cleared)	Tab/session closes	Set manually (can persist or expire)
Sent to server?	No	No	Yes — sent with every HTTP request
Accessible via JS	<code>localStorage.setItem()</code>	<code>sessionStorage.setItem()</code>	<code>document.cookie</code>
Common MERN use	Storing JWT tokens (caution: XSS risk), theme/preferences	Temporary form data, one-time flags	Session IDs, auth tokens with <code>httpOnly</code> flag (safer than localStorage against XSS)

```
// Save & read data
localStorage.setItem('token', jwt);
const token = localStorage.getItem('token');
localStorage.removeItem('token');
```

Interview Q: "Where should a JWT be stored — localStorage or an httpOnly cookie?" **A:** `httpOnly` cookies are generally safer because JavaScript cannot read them, protecting against XSS attacks; localStorage is readable by any script on the page, making it vulnerable if the site has an XSS hole. This is a very common MERN-stack security question.

11.2 Other Important HTML5 APIs

API	Purpose
<code><canvas></code>	Pixel-based drawing surface for graphics/games, controlled via JS (<code>getContext('2d')</code>)
<code><svg></code>	Vector-based graphics, resolution-independent, DOM-addressable (unlike canvas)
Geolocation API	<code>navigator.geolocation.getCurrentPosition()</code> — gets user's location (requires HTTPS + permission)
Drag & Drop API	<code>draggable="true"</code> + <code>ondragstart/ondrop</code> events — native drag-and-drop UI
Web Workers	Run JS in a background thread, doesn't block UI — for heavy computation
Service Workers	Enable offline support, caching, push notifications (foundation of PWAs)
History API	<code>pushState/replaceState</code> — enables client-side routing without full reload (basis of React Router)
Fetch API	Modern promise-based way to make HTTP requests (replaced XMLHttpRequest) — core of connecting React frontend to Express/Node backend

MERN link: `fetch()` or `axios` calls from React talk to Express REST APIs; the History API is what React Router uses under the hood for client-side navigation without page reloads.

11.3 Canvas vs SVG

Aspect	Canvas	SVG
Type	Pixel/raster based (bitmap)	Vector based (XML markup)
DOM	Single element, no access to drawn shapes individually	Each shape is its own DOM node — inspectable, animatable via CSS
Scaling	Loses quality when scaled	Infinitely scalable, no quality loss
Best for	Games, image editing, real-time graphics with many objects	Icons, charts, logos, diagrams

12 Accessibility (a11y) & ARIA

12.1 Why Accessibility Matters

Accessible HTML ensures the site is usable by people with visual, auditory, motor, or cognitive disabilities (screen readers, keyboard-only navigation). It's also a legal requirement in many countries and directly impacts SEO.

12.2 Core Rules

- Use **semantic tags** instead of `div`s wherever possible (browsers give them a built-in accessibility role for free).
- Every `<img>` needs meaningful `alt` text (empty `alt=""` for purely decorative images).
- Every form input needs an associated `<label>`.
- Maintain a logical heading hierarchy (`h1` → `h2` → `h3`, don't skip levels).
- Ensure sufficient color contrast (WCAG AA: 4.5:1 for normal text).
- All interactive elements must be reachable and operable via keyboard (Tab, Enter, Space).

12.3 ARIA (Accessible Rich Internet Applications)

ARIA attributes add accessibility info when native HTML semantics aren't enough (e.g. custom components like a React-built dropdown or modal).

Attribute	Use
<code>role="button"</code>	Tells screen readers an element acts as a button (e.g. a clickable <code><div></code> — though a real <code><button></code> is always preferred)
<code>aria-label="..."</code>	Provides an accessible name when no visible text exists (icon-only buttons)
<code>aria-hidden="true"</code>	Hides purely decorative elements from screen readers
<code>aria-expanded="true/false"</code>	State of a collapsible menu/accordion
<code>aria-live="polite"</code>	Announces dynamically updated content (e.g. a live cart total in a React app)

Golden rule (often quoted in interviews): "No ARIA is better than bad ARIA" — always prefer a native semantic element over adding ARIA roles to a generic `div`.

13 Entities, Comments & Special Characters

Entity	Renders As	Meaning
<code>&lt;</code>	<	Less than (needed since < starts a tag)
<code>&gt;</code>	>	Greater than
<code>&amp;</code>	&	Ampersand
<code>&quot;</code>	"	Double quote
<code>&nbsp;</code>	(space)	Non-breaking space — prevents line break / collapses like normal space otherwise
<code>&copy;</code>	©	Copyright symbol

Entities are needed because characters like `<` have special meaning in HTML syntax — writing them raw would break parsing.

14 HTML in React (JSX) — Key Differences

This is one of the most important chapters for a **MERN developer interview** — you must clearly explain how JSX differs from plain HTML.

Plain HTML	JSX (React)	Why
<code>class="btn"</code>	<code>className="btn"</code>	<code>class</code> is a reserved JS keyword
<code>for="email"</code> (on label)	<code>htmlFor="email"</code>	<code>for</code> is a reserved JS keyword
<code>onclick="fn()"</code>	<code>onClick={fn}</code>	Event handlers are camelCase and take a JS function reference, not a string
<code><input></code>	<code><input /></code>	JSX requires ALL tags (including void elements) to be explicitly closed
<code>style="color:red"</code> (string)	<code>style={{color: 'red'}}</code>	JSX style prop takes a JS object with camelCase CSS properties
<code>tabindex="1"</code>	<code>tabIndex={1}</code>	Most multi-word attributes become camelCase in JSX
Multiple root elements allowed	Must return ONE root element (or a <code><>...</></code> Fragment)	JSX compiles to a single <code>React.createElement()</code> call tree
Comments: <code><!-- ... --></code>	<code>/* ... */</code>	JSX comments must be inside curly braces (they're JS expressions)
Conditional rendering: none natively	<code>{condition && <p>Shown</p>}</code>	Logic is expressed with JS expressions inside <code>{}</code>

```
// JSX example
function Button() {
  return (
    <button className="primary" onClick={() => alert('Clicked!')}>
      Click Me
    </button>
  );
}
```

Interview Q: "Is JSX HTML?" **A:** No — JSX is syntactic sugar that compiles (via Babel) into `React.createElement()` calls, which build the Virtual DOM. It only *looks* like HTML but is actually JavaScript.

15 Performance & Best Practices (2026)

- Use **semantic tags** — improves SEO, accessibility, and code readability instantly.
- Always set `width / height` on images/video to avoid layout shift (Core Web Vitals: CLS).
- Use `loading="lazy"` for below-the-fold images and iframes.
- Minify HTML/CSS/JS in production builds (Vite/Webpack does this automatically in MERN projects).
- Use `defer` for scripts that touch the DOM; avoid blocking render with synchronous scripts in `<head>`.
- Serve modern image formats (WebP/AVIF) with `<picture>` fallback.
- Only one `<h1>` per page; maintain heading hierarchy.
- Validate HTML (W3C validator) to catch unclosed tags / invalid nesting.
- Keep DOM depth reasonable — deeply nested trees slow down layout/reflow.
- Use `rel="noopener noreferrer"` on all `target="_blank"` links.
- Prefer native elements (`<button>` , `<dialog>` , `<details>`) over reinventing them in JS — less code, built-in accessibility.

16 Master Differentiation Tables

16.1 HTML4 vs HTML5

HTML4	HTML5
No native audio/video (needed Flash)	Native <code><audio></code> / <code><video></code>
No semantic tags (only divs)	header, nav, section, article, footer, etc.
No client storage besides cookies	localStorage, sessionStorage, IndexedDB
Doctype: long, complex	Doctype: simply <code><!DOCTYPE html></code>
No canvas/SVG support built-in	Native <code><canvas></code> and inline <code><svg></code>
No native form validation	required, pattern, type=email etc.

16.2 HTML vs XHTML

HTML	XHTML
Case-insensitive tags	Must be lowercase
Some tags can be unclosed (<code></code>)	Every tag must be explicitly closed
Attribute values can be unquoted	Attribute values must always be quoted
Lenient parser (forgives errors)	Strict XML parser (breaks page on error)

16.3 div vs span

div	span
Block-level	Inline-level
Used to group larger sections/layout blocks	Used to wrap small inline text/phrases for styling
Starts on new line	Stays in the text flow

16.4 id vs class — see Chapter 3.5

16.5 GET vs POST — see Chapter 8.2

16.6 async vs defer — see Chapter 9.3

16.7 <script> placement: head vs end of body

In <code><head></code> (no defer)	End of <code><body></code>	<code><head></code> + defer (modern default)
Blocks HTML parsing → slow first paint	DOM ready before script runs; simple, widely used pre-ES6	Non-blocking download + runs after DOM ready — best of both, also visible early in the head

17 Interview Question Bank

1 What is the difference between HTML and HTML5?

HTML5 added semantic tags, native audio/video, canvas/SVG, Web Storage APIs, offline support, native form validation, and removed dependency on plugins like Flash. See table 16.1.

2 What happens if you don't declare DOCTYPE?

The browser renders in "Quirks Mode" — an older, inconsistent rendering mode that emulates legacy browser bugs, causing unpredictable CSS box-model behaviour across browsers.

3 Why is the `alt` attribute important?

Accessibility (screen readers read it), fallback text if image fails to load, and SEO (image search indexing).

4 What are void elements? Name five.

Self-closing elements with no content/children: `img`, `br`, `hr`, `input`, `meta` (also `link`, `source`, `area`, `col`).

5 Difference between `<script>`, `<script async>`, and `<script defer>`?

See detailed diagram & table in Chapter 9.3 — plain scripts block parsing, `async` runs as soon as fetched (unordered), `defer` runs after parsing completes (ordered).

6 What's the difference between `localStorage`, `sessionStorage`, and cookies?

See table in Chapter 11.1 — key differences are capacity, expiry, and whether data is auto-sent to the server (cookies only).

7 How do you make a website responsive using only HTML?

Add `<meta name="viewport" content="width=device-width, initial-scale=1.0">`, use relative units, and `<picture>` / `srcset` for responsive images (actual responsiveness is completed with CSS media queries).

8 What is semantic HTML and why does it matter for SEO?

Tags that convey meaning (nav, article, header) rather than pure styling hooks (div). Search engines weight content inside meaningful tags more heavily and can better understand page structure, improving ranking and accessibility.

9 What is the difference between `<strong>` / `<b>` and `<em>` / `<i>`?

`strong/em` carry semantic meaning for accessibility tools; `b/i` are purely visual with no added meaning.

10 What is the purpose of the `rel="noopener noreferrer"` attribute?

Security — prevents a page opened via `target="_blank"` from accessing `window.opener` (protects against "reverse tabnabbing") and hides referrer data.

11 Difference between block, inline, and inline-block elements?

See table in Chapter 3.3 — block takes full width and starts a new line; inline flows with text and ignores vertical margin/width; inline-block flows inline but respects box-model sizing.

12 What is the DOM?

A tree-like, in-memory object representation of the HTML document that JavaScript can read/manipulate. Every HTML tag becomes a "node" (see Fig 2.1).

13 What's the difference between `<div>` and `<section>`?

`section` is semantic (a thematic content group, usually with its own heading); `div` is a generic, meaningless container used only for styling/layout.

14 How do you optimize images for web performance?

Use appropriately sized images, modern formats (WebP/AVIF) via `<picture>`, `loading="lazy"`, set width/height to reserve layout space, and compress before upload.

15 What is the difference between GET and POST forms?

See table in Chapter 8.2 — GET sends data via URL (visible, cacheable, idempotent); POST sends data in the request body (hidden, used for creating/mutating data).

16 What is the difference between `id` and `class` in HTML/CSS specificity?

`id` must be unique and has higher CSS specificity (100) than `class` (10), which can be reused across multiple elements.

17 What does `enctype="multipart/form-data"` do and when do you need it?

It's required whenever a form uploads files, since regular URL-encoded forms cannot carry binary data.

18 How does JSX differ from HTML?

See full table in Chapter 14 — `className` vs `class`, `camelCase` event handlers/attributes taking function/object values, mandatory self-closing tags, single root element, and JS-expression comments.

19 What is Cumulative Layout Shift (CLS) and how does HTML help prevent it?

CLS is a Core Web Vital measuring unexpected layout jumps. Setting explicit width/height on images/video/ads reserves their space before they load, preventing the page from jumping.

20 What is ARIA and when should you use it?

Accessible Rich Internet Applications attributes add accessibility semantics missing from custom-built components (like a React modal). Rule: prefer native semantic HTML first; only add ARIA when no native element fits.

21 What's the difference between `<canvas>` and `<svg>`?

See table in Chapter 11.3 — `canvas` is pixel-based and loses quality on scale (good for games); `SVG` is vector-based, DOM-addressable, and scales without quality loss (good for icons/charts).

22 Why should you avoid using tables for page layout?

It's semantically incorrect (tables mean tabular data, not layout), harms accessibility (screen readers announce rows/columns unnecessarily), and is inflexible for responsive design — CSS Grid/Flexbox should be used instead.

23 What is the difference between `<link>` and `<a>` ?

`<link>` is used inside `<head>` to connect external resources (stylesheets, favicons) and is not visible/clickable; `<a>` creates a visible, clickable hyperlink in the body.

24 What is the significance of the `lang` attribute on `<html>`?

Helps screen readers use correct pronunciation rules and assists browsers/SEO tools with automatic translation and language-specific behaviour.

25 How would you make an icon-only button accessible?

Add `aria-label="Descriptive action"` to the button so screen readers announce its purpose even without visible text.

26 What is the difference between `<meta charset="UTF-8">` omission risk?

Without it, browsers may guess encoding incorrectly, causing special characters/emojis/non-English text to render as garbled symbols ("mojibake").

27 Explain progressive enhancement vs graceful degradation.

Progressive enhancement: build a basic working HTML experience first, then layer CSS/JS enhancements for capable browsers. Graceful degradation: build the full-featured experience first, then ensure it still works (in reduced form) on older/less capable browsers.

28 What is the purpose of the `<noscript>` tag?

Displays fallback content when JavaScript is disabled or unsupported in the user's browser.

29 How do web fonts get loaded in HTML and what performance issue can they cause?

Via `<link rel="stylesheet">` (e.g. Google Fonts) or `@font-face` in CSS. They can cause FOIT/FOUT (flash of invisible/unstyled text) while downloading — mitigated with `font-display: swap`.

30 What is the difference between `<article>` and `<section>` ?

article is meant for standalone, independently distributable content (could be syndicated/reused on its own, like a blog post); section groups related content within a page/article that isn't necessarily independent.

31 What does the `sandbox` attribute on an `iframe` do?

Restricts the embedded page's capabilities (disabling scripts, form submission, popups, top-navigation) by default — a key security control when embedding third-party/untrusted content.

32 What's the difference between synchronous and asynchronous form submission in a MERN app?

A plain HTML form submit (no JS) triggers a full page reload/navigation to the action URL. In React, form submission is typically intercepted with `e.preventDefault()` and sent asynchronously via `fetch` / `axios` to the Express API, updating the UI without a reload (SPA behaviour).

33 Why is inline CSS (`style="..."`) discouraged?

Highest CSS specificity (hard to override), not reusable, bloats HTML, and separates concerns poorly compared to external stylesheets or CSS-in-JS/modules used in React apps.

34 What is the difference between `<b>` / `<strong>` for SEO purposes?

Search engines give slight extra semantic weight to content wrapped in `strong/em` since it signals genuine importance, unlike purely stylistic `b/i`.

35 How does the browser handle unknown/custom HTML tags?

Unknown tags are treated as anonymous inline elements and still added to the DOM (this is the basis for Web Components / custom elements like `<my-widget>`) — the browser doesn't reject them outright, it just applies no special styling/behaviour.

18 Quick Revision Cheat-Sheet

18.1 One-Page Summary

Topic	Remember This
Doctype	<code><!DOCTYPE html></code> → triggers Standards Mode
Structure	html → head (metadata) + body (visible content)
Void tags	img, br, hr, input, meta, link (no closing tag)
Semantic layout	header → nav → main (article/section) + aside → footer
Block vs inline	Block = new line + full width; Inline = flows with text
Forms	label needs matching <code>for</code> / <code>id</code> ; GET=URL data, POST=body data
Images	Always set alt + width/height; use loading="lazy"
Scripts	Prefer <code>defer</code> for app scripts touching the DOM
Storage	localStorage = permanent; sessionStorage = tab-only; cookies = sent to server
Accessibility	Semantic tags first, ARIA only when necessary, alt text always
JSX vs HTML	className, htmlFor, camelCase events, self-close everything, one root element
Security	rel="noopener noreferrer" on target="_blank"; sandbox on untrusted iframes; httpOnly cookies over localStorage for tokens

Final tip for interviews: Whenever asked "what's the difference between X and Y," structure your answer as: (1) one-line definition of each, (2) 2-3 concrete differences, (3) when to use which. This structure alone impresses most interviewers.